Scope of Work: AI-Assisted Product Catalog Management Platform

1. Document Purpose

This document defines the Scope of Work for a Proof of Concept (POC) and a future production solution for AI-assisted catalog creation, editing, translation, approval, versioning, and print readiness for a large machine tool manufacturer.

It is written so business stakeholders, delivery managers, designers, and developers can align on what needs to be built, what should be demonstrated next week, and how the technical team should approach it.

2. Customer Background

The client is a major Indian machine tool manufacturer with multiple product lines, including:

- CNC Turning Machines
- Vertical Machining Centers (VMC)
- Horizontal Machining Centers (HMC)
- CNC Grinding Machines
- Turn-Mill Centers

Each product line has catalogs in digital and printed format. These catalogs contain a mix of:

- product photographs and branding
- specifications tables
- machine features
- standard and optional accessories
- technical diagrams
- footers, legal notes, QR codes, and contact information

The sample catalog reviewed for this scope is a 2-page A4 brochure with a branded cover page and a specification-heavy second page.

3. Business Problem

The current process is manual and distributed across multiple teams. When a machine specification, image, feature, accessory list, or diagram changes:

- multiple people need to be informed
- updates can be delayed or missed
- digital and printed versions can go out of sync

- language support is limited because content is primarily maintained in English
- approval and print release are not tightly controlled in one workflow
- older and newer versions are difficult to track consistently

The client wants a system where authorized users can request catalog changes in natural language, optionally attach a change document such as Excel or Word, and have AI prepare an updated version for human review before release.

4. Business Objectives

The solution should:

- reduce time required to update catalogs
- reduce dependency on multiple teams for routine updates
- ensure only approved versions are released for digital or print use
- support multilingual digital catalogs, especially Indian languages
- maintain brand consistency and professional design quality
- support multiple templates such as 2-page, 3-page, and other catalog layouts
- maintain version history and change traceability
- support future integration with the client's design system or design repository

5. High-Level Vision

The target solution is an AI-assisted catalog operations platform, not just a PDF editor.

The platform should let a business user:

1. select a product catalog or template
2. enter a prompt such as "change spindle speed to 4500 rpm, replace product image, and generate Hindi version"
3. optionally upload a supporting change document such as Excel, PDF, or Word
4. review AI-suggested edits in a visual comparison view
5. approve or reject the changes
6. publish a new digital version and optionally release a print-ready version

6. Scope of Work

6.1 In Scope for Full Solution

- Catalog ingestion and management for multiple machine families
- Template-based catalog generation and editing
- Prompt-based editing through a chat-style user interface
- Structured editing of specifications, features, accessories, disclaimers, and contact blocks

- Asset replacement such as product image updates
- Multilingual catalog generation for major Indian languages
- Human approval workflow before digital release or print release
- Version history with rollback and audit trail
- Sync detection from design system or approved content source
- Export of approved digital and print-ready output
- Template upload and template creation assistance

6.2 In Scope for Immediate POC

- A working web UI with a prompt box
- Loading one sample catalog based on the provided brochure
- Editing a selected set of content areas through prompt instructions
- Showing before/after preview of the catalog
- Exporting an updated PDF
- Basic version save for each prompt-driven revision
- Optional translation of selected content into one Indian language, preferably Hindi

6.3 Out of Scope for Immediate POC

- Support for every existing catalog variation
- Full automatic editing of arbitrary PDFs with no template preparation
- Full ERP/PLM/PIM integration
- Full print production workflow integration with external vendors
- Final enterprise security hardening and SSO rollout
- Full multilingual QA for all Indian languages

7. Key Functional Requirements

7.1 Catalog and Template Management

- The system shall support catalog templates such as 2-page, 3-page, and future custom layouts.
- The system shall allow an authorized user to upload a new template.
- The system shall support AI-assisted template creation for new catalog formats.
- The system shall maintain template metadata, version, owner, and status.
- The system shall separate reusable design layout from editable business content.

7.2 Product Content Management

- The system shall store product content in structured form where possible, including:
    - product name

    - machine category
    - hero image
    - feature list
    - standard accessories
    - optional accessories
    - specification table
    - legal notes
    - contact and footer content
- The system shall allow manual and AI-assisted updates to this content.
- The system shall support import of change inputs from Excel, CSV, PDF, or Word attachments.

7.3 Prompt-Based Editing

- The system shall provide a chat-like interface for change requests.
- The system shall understand prompts such as:
    - "Change spindle speed from 4000 rpm to 4500 rpm"
    - "Replace the machine image with the new image"
    - "Add MachineConnect to smart features"
    - "Create a Hindi digital version"
    - "Use the 3-page template and move diagrams to page 3"
- The system shall identify impacted content blocks before applying changes.
- The system shall generate a reviewable draft rather than auto-publishing changes.

7.4 Multilingual Support

- The system shall support English as the source language.
- The system shall support multilingual output for selected Indian languages.
- Priority languages for roadmap discussion should include:
    - Hindi
    - Tamil
    - Telugu
    - Kannada
    - Marathi
    - Gujarati
    - Bengali
    - Malayalam
- The system shall preserve layout quality across languages as far as possible.
- The system shall flag overflow or layout breakage introduced by translation.

7.5 Approval Workflow

- The system shall require human approval before final release.
- The system shall support at least draft, in review, approved, rejected, and released states.
- The system shall show a clear difference between the current approved version and the proposed version.
- The system shall record who requested, reviewed, approved, rejected, or released each version.

7.6 Versioning and Audit Trail

- The system shall create a new version for every approved update.
- The system shall preserve previous versions for comparison and rollback.
- The system shall store change summaries and timestamps.
- The system shall support side-by-side comparison of versions.

7.7 Design System and Asset Sync

- The future solution shall support integration with an approved design source such as Figma, Adobe workflow, DAM, or another internal design repository.
- The system shall detect relevant asset or template changes and create a pending update recommendation.
- The system shall not auto-release design changes without human approval.
- The system shall keep a mapping between content fields and layout elements.

7.8 Output Generation

- The system shall generate digital PDF output.
- The system shall generate print-ready output after approval.
- The system shall preserve page size, branding, quality, and alignment.
- The system shall support downloadable version packages with metadata.

8. Important Solution Principle

For the production solution, the system should not rely on directly editing arbitrary PDFs pixel-by-pixel as the main strategy.

Instead, the recommended approach is:

- treat the catalog as a structured content model plus a controlled layout template
- generate the final PDF from approved structured content and brand templates
- use PDF parsing only for import, migration, comparison, and limited extraction

This is important because direct PDF editing is fragile for tables, diagrams, multilingual text expansion, and consistent print quality.

9. Sample Catalog Observations That Influence Design

Based on the sample 2-page catalog:

- Page 1 contains brand-heavy layout, hero product image, marketing header, and three grouped bullet sections.
- Page 2 contains technical diagrams, a dense specifications table, charts, contact blocks, and print disclaimer text.
- Some elements are simple text replacements.
- Some elements are structured tabular updates.
- Some elements are visual assets that require layout-aware handling.
- Translation will increase text length and may require dynamic font sizing or block resizing.
- The PDF metadata indicates the sample was likely exported from CorelDRAW 2021, so access to source design files could significantly reduce effort for migration and template recreation.

This means the product must distinguish between:

- editable text blocks
- structured data blocks
- locked brand elements
- visual assets
- diagram or chart regions that may need manual review

10. Proposed Solution Architecture

10.1 Recommended Components

- Frontend web application
- Backend API service
- Catalog template engine
- AI orchestration layer
- Document ingestion and extraction service
- Asset repository
- Versioning and audit service
- Approval workflow service
- PDF rendering service

10.2 Recommended Logical Flow

1. User selects an existing catalog or template.
2. User enters a prompt and optionally uploads change documents.
3. Backend extracts requested changes and maps them to structured fields and affected layout zones.

4. AI proposes updates.
5. Rendering service creates a draft preview.
6. User reviews visual diff and structured diff.
7. User approves or rejects.
8. Approved version is stored, versioned, and exported as PDF.

10.3 Suggested Data Model

At minimum, define these entities:

- Product
- Catalog
- CatalogVersion
- Template
- TemplateVersion
- ContentBlock
- Asset
- ChangeRequest
- Approval
- TranslationVariant
- AuditEvent

Each content block should ideally have:

- unique ID
- type such as heading, bullet_list, spec_table, image, footer, diagram
- source content
- translated content
- page mapping
- template mapping
- lock status

11. Technical Guidance for the Developer Team

11.1 Build Strategy

For the POC, build a deterministic template-driven web application and use AI only for:

- understanding the user prompt
- extracting requested changes from attachments
- mapping natural language to structured edit operations

- generating translation drafts
- generating change summaries

Do not let the AI freely redraw the catalog without template constraints.

11.2 Recommended POC Approach

Use the sample catalog to create one editable template in HTML/CSS or another layout system that can render reliably to PDF.

Recommended workflow:

1. Recreate the sample catalog as an editable template.
2. Store its content in JSON.
3. Build a prompt interface.
4. Convert prompt to structured commands such as:
    - update_spec
    - replace_text
    - replace_image
    - translate_catalog
5. Re-render the catalog preview.
6. Export updated PDF.

This will let the team show "edit existing catalog by prompt" in a credible way next week without promising universal PDF editing.

11.3 Recommended Technology Stack for POC

- Frontend: React or Next.js
- Backend: Python FastAPI or Node.js
- Rendering: HTML/CSS to PDF using Playwright or Puppeteer
- Storage: PostgreSQL or SQLite for POC, file storage for assets
- AI layer: LLM-based prompt parser and translation helper
- Versioning: database revision history plus generated PDF snapshots

11.4 Why HTML/CSS Template Rendering Is Preferred for POC

- faster to build than a true desktop publishing engine
- easier to preview in browser
- easier to support prompt-driven structured edits
- easier to support multiple templates
- easier to debug than direct PDF patching

11.5 Handling Existing PDFs

For POC, treat the current PDF as:

- source reference for layout recreation
- source for text extraction
- source for screenshot comparison

Do not promise perfect editing of any arbitrary PDF file in the first demo.

For later phases, support:

- PDF import and segmentation
- semi-automatic template mapping
- migration of legacy catalogs into the structured template system

11.6 Handling Translation

Translation should be done at content block level, not at final PDF level.

Recommended approach:

- translate structured source content
- render translated content into the template
- run overflow checks
- flag blocks requiring manual correction

11.7 Handling Design System Sync

For the POC, mock design sync with a simple controlled source such as:

- a JSON design token file
- a watched asset folder
- a manually triggered "refresh template assets" action

For the production roadmap, build connectors to the actual design system used by the client.

11.8 Using Codex Effectively

Codex should be used as a development accelerator, not as the production runtime itself.

Recommended use of Codex by the developer:

- scaffold frontend and backend modules
- generate template code from the sample catalog
- create edit-operation handlers
- build diff and approval UI

- generate test cases for prompt-to-edit workflows
- speed up integration and refactoring

The deployed product should remain a normal application with maintainable source code, tests, and controlled APIs.

12. POC Definition for Next Week

12.1 POC Objective

Demonstrate that a user can update a machine catalog through a prompt-based UI and receive a visually updated draft for approval.

12.2 Recommended POC Scope

Build support for one product family and one 2-page catalog template based on the sample brochure.

The POC should demonstrate:

- opening the sample catalog
- entering a change prompt
- updating at least:
  - product title
  - one or more specification values
  - one feature or accessory list item
  - one image or placeholder image
- generating updated preview
- saving a new version
- exporting updated PDF
- optionally creating one translated version in Hindi

12.3 Sample Demo Scenarios

Scenario A:

- Prompt: "Update maximum spindle speed from 4000 rpm to 4500 rpm, change standard chuck size to 200 mm, and add high pressure coolant system under optional accessories."

Scenario B:

- Prompt: "Create Hindi version of this catalog and keep the same layout as far as possible."

Scenario C:

- Prompt: "Use the new product image and update the hero title to J 300 LM Plus."

12.4 POC Acceptance Criteria

- User can view the sample catalog in the UI.
- User can submit a prompt.
- System interprets the prompt into visible change operations.
- System generates updated preview within an acceptable demo time.
- User can review old versus new version.
- User can approve and export updated PDF.
- At least basic revision history is recorded.

13. POC Work Packages and Deliverables

13.1 Work Packages

1. Template Recreation

- recreate the sample 2-page catalog as an editable template
- identify locked brand elements versus editable content zones

2. Content Modeling

- define JSON schema for title, image, bullet lists, specs, footer, and metadata
- map the sample catalog into structured content blocks

3. Prompt-to-Edit Engine

- convert user prompt into structured update operations
- support value replacement, list update, image swap, and translation request

4. Preview and Export

- render browser preview
- render updated PDF for download

5. Versioning and Approval

- save prompt request and generated revision
- provide basic approve action for demo purposes

6. Demo Readiness

- prepare sample prompts
- prepare one or two sample images and one optional change sheet
- validate end-to-end demo flow

13.2 POC Deliverables

- working POC web application

- one editable 2-page sample catalog template
- prompt-based edit flow
- updated PDF export
- basic version history
- short developer setup guide
- short demo script for presentation

14. Recommended POC UI

The POC UI should have:

- catalog preview panel
- prompt input panel
- attachment upload option
- detected changes summary
- version history panel
- approve and export actions

Ideal screen layout:

- left side: existing and updated preview
- right side: prompt box, attachments, extracted changes, approve button

15. Non-Functional Requirements

- Professional output quality suitable for client demonstration
- Clear separation between draft and approved versions
- Basic role-based access for editor and approver in later phases
- Reasonable performance for 2 to 5 page catalogs
- Secure handling of uploaded files
- Full auditability of change actions
- Extensible architecture for future template and language expansion

16. Risks and Constraints

- Arbitrary PDF editing is unreliable for production use without template mapping.
- Translation can cause text overflow and layout changes.
- Technical diagrams may need manual review even after automated updates.
- True print production standards may require additional review in later phases.
- Final design system integration depends on the actual source platform used by the client.

17. Assumptions

- The client can provide editable source assets for images and branding.
- The client can identify authoritative owners for specifications and accessory lists.
- The client can confirm which design system or design repository is the system of record.
- Human review remains mandatory before print release.
- Initial POC can focus on one sample catalog and one language extension.

18. Suggested Delivery Phases

Phase 1: POC

- Single sample catalog
- Single editable template
- Prompt-based update flow
- Preview, approval, export
- Basic versioning

Phase 2: Pilot

- Multiple templates
- Multiple products
- Translation workflow
- Better asset handling
- Better approval and role controls

Phase 3: Production

- Integration with design source and enterprise systems
- Scalable template governance
- Full audit, versioning, and release workflow
- Broader language coverage
- Print production controls

19. Open Questions for Client Confirmation

- What is the master source of truth for machine specifications today?
- Which system is considered the design system of record?
- Are catalogs currently designed in CorelDRAW, Adobe, InDesign, Figma, or another tool?
- Can the client share editable source files such as CorelDRAW, InDesign, fonts, and linked images for the sample catalog?

- Which languages are highest priority for rollout after English?
- Who are the approvers for digital release and print release?
- Are diagrams and power charts manually created or generated from engineering data?

20. Delivery Recommendation

For next week's presentation, position the POC as:

- a smart catalog editing assistant
- template-driven and approval-led
- capable of prompt-based structured edits
- capable of multilingual expansion
- designed to evolve into a governed catalog publishing platform

This is a stronger and more believable story than positioning it as a generic AI that edits any PDF automatically.

21. Summary

The recommended solution is an AI-assisted catalog management platform with:

- prompt-based editing
- template-driven rendering
- multilingual support
- approval workflow
- version control
- future design system sync

The recommended next-week POC is a focused demonstration on one sample 2-page catalog showing prompt-based edits, preview, approval, versioning, and PDF export.